Status	Finished
Started	Wednesday, 28 October 2026, 6:16 PM
Completed	Wednesday, 28 October 2026, 6:19 PM
Duration	3 mins 13 secs
Grade	**12.00** out of 15.00 (**80**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Which of the following statements about linear search is true?

a. Linear search divides the list into halves.
b. Linear search is more efficient than binary search on large lists.
c. Linear search can be applied to both sorted and unsorted lists. ✓
d. Linear search requires the list to be sorted.

Question **2**

Correct

Mark 1.00 out of 1.00

In the context of searching, what is a successful search?

a. When the element is found in the list ✓
b. When the search algorithm finishes
c. When the list is sorted
d. When the list contains duplicate elements

Question **3**

Correct

Mark 1.00 out of 1.00

In binary search, what happens if the middle element does not match the target element?

a. The search stops
b. The search continues in the left or right sublist ✓
c. The list is sorted
d. The search continues from the beginning

Question **4**

Incorrect

Mark 0.00 out of 1.00

What happens in a binary search if the list has an even number of elements?

a. The higher middle element is chosen as the middle element
b. The search stops
c. The middle element is chosen randomly
d. The lower middle element is chosen as the middle element ✗

Question **5**

Correct

Mark 1.00 out of 1.00

In _____ checks the elements of a list, one at a time, without skipping any element.

a. Both (1) & (3)
b. Binary search
c. Linear search ✓
d. Hash search

Question **6**

Correct

Mark 1.00 out of 1.00

Which method of searching involves sequentially comparing each element until a match is found?

a. Linear search ✓
b. Jump search
c. Binary search
d. Hashing

Question **7**

Correct

Mark 1.00 out of 1.00

In which situation is linear search more efficient than binary search?

a. When the list is small and sorted
b. When the list is large and unsorted
c. When the list is large and sorted
d. When the list is small and unsorted ✓

Question **8**

Correct

Mark 1.00 out of 1.00

What is the best-case time complexity of linear search?

a. O👎
b. O(n log n)
c. O(1) ✓
d. O(log n)

Question **9**

Incorrect

Mark 0.00 out of 1.00

Given an array arr = {45,77,89,90,94,99,100} and key = 99; what are the mid values(corresponding array elements) in the first and second levels of recursion?

a. 90 and 99
b. 90 and 94
c. 89 and 99 ✗
d. 89 and 94

Question **10**

Correct

Mark 1.00 out of 1.00

Which of the following is a limitation of binary search?

a. It can only be applied to large lists
b. It requires the list to be sorted ✓
c. It does not work with negative numbers
d. It is slower than linear search for small lists

Question **11**

Incorrect

Mark 0.00 out of 1.00

What type of search would be most appropriate for finding an element in a list that is frequently updated?

a. Interpolation search
b. Linear search
c. Hash search ✗
d. Binary search

Question **12**

Correct

Mark 1.00 out of 1.00

Which of the following scenarios is best suited for applying binary search?

a. When the list contains duplicate elements
b. When the list is unsorted
c. When the list is sorted ✓
d. When the list is very small

Question **13**

Correct

Mark 1.00 out of 1.00

What is searching in the context of computer science?

a. Determining whether an element is present in a list ✓
b. Deleting elements from a list
c. Sorting elements in a list
d. Inserting elements into a list

Question **14**

Correct

Mark 1.00 out of 1.00

In which type of search is the list divided into smaller sublists during the search process?

a. Sequential search
b. Binary search ✓
c. Linear search
d. Hash search

Question **15**

Correct

Mark 1.00 out of 1.00

The average case occurs in the linear search algorithm

a. When the item is not the array at all
b. When the item is somewhere in the middle of the array ✓
c. When the item is the last element in the array
d. Item is the last element in the array or item is not there at all

Status	Finished
Started	Wednesday, 28 October 2026, 6:20 PM
Completed	Wednesday, 28 October 2026, 6:31 PM
Duration	11 mins 12 secs
Grade	**9.00** out of 10.00 (**90**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Balanced strings are those that have an equal quantity of 'L' and 'R' characters.

Given a balanced string s, split it in the maximum amount of balanced strings.

Return the maximum amount of split balanced strings.

Example 1:

Input:

RLRRLLRLRL

Output:

4

Explanation: s can be split into "RL", "RRLL", "RL", "RL", each substring contains same number of 'L' and 'R'.

Example 2:

Input:

RLLLLRRRLR

Output:

3

Explanation: s can be split into "RL", "LLLRRR", "LR", each substring contains same number of 'L' and 'R'.

Example 3:

Input:

LLLLRRRR

Output:

1

Explanation: s can be split into "LLLLRRRR".

Constraints:

1 <= s.length <= 1000

s[i] is either 'L' or 'R'.

s is a balanced string.

For example:

Test	Result
`print(BalancedStrings('RLRRLLRLRL'))`	4
`print(BalancedStrings('RLLLLRRRLR'))`	3

Answer: (penalty regime: 0 %)

Reset answer

```
def BalancedStrings(s):
    count = 0
    left = 0
    right = 0

    for char in s:
        if char == 'L':
            left += 1
        elif char == 'R':
            right += 1

        if left == right:
            count += 1

    return count
```

	Test	Expected	Got	
✓	`print(BalancedStrings('RLRRLLRLRL'))`	4	4	✓
✓	`print(BalancedStrings('RLLLLRRRLR'))`	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given two Strings s1 and s2, remove all the characters from s1 which is present in s2.

Constraints

1<= string length <= 200

Sample Input 1

experience
enc

Sample Output 1

xpri

Answer: (penalty regime: 0 %)

```
import sys

def main():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) >= 2:
        s1 = input_data[0]
        s2 = input_data[1]
        s2_set = set(s2)
        result = "".join([char for char in s1 if char not in s2_set])
        print(result)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	experience enc	xpri	xpri	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

An list contains N numbers and you want to determine whether two of the numbers sum to a given number K. For example, if the input is 8, 4, 1, 6 and K is 10, the answer is yes (4 and 6). A number may be used twice.

Input Format

The first line contains a single integer n , the length of list

The second line contains n space-separated integers, list[i].

The third line contains integer k.

Output Format

Print Yes or No.

Sample Input

7

0 1 2 4 6 5 3

1

Sample Output

Yes

For example:

Input	Result
5 8 9 12 15 3 11	Yes
6 2 9 21 32 43 43 1 4	No

Answer: (penalty regime: 0 %)

```
import sys

def main():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    lst = [int(x) for x in input_data[1:n+1]]
    k = int(input_data[n+1])

    seen = set(lst)
    found = False

    for x in lst:
        if k - x in seen:
            found = True
            break

    if found:
        print("Yes")
    else:
        print("No")

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	5 8 9 12 15 3 11	Yes	Yes	✓
✓	6 2 9 21 32 43 43 1 4	No	No	✓
✓	6 13 42 31 4 8 9 17	Yes	Yes	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

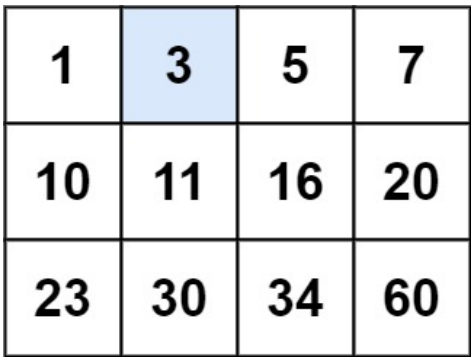
You are given an `m x n` integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `True` *if* `target` *is in* `matrix` *or* `False` *otherwise*.

You must write a solution in `O(log(m * n))` time complexity.

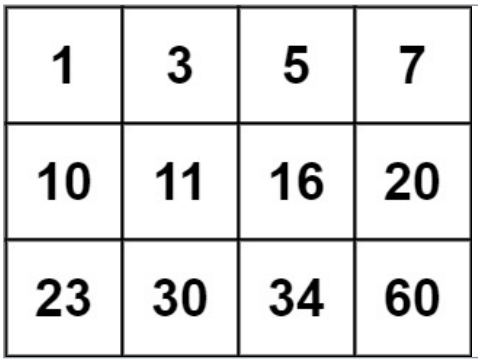
Example 1:



1	3	5	7
10	11	16	20
23	30	34	60

```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: True
```

Example 2:



1	3	5	7
10	11	16	20
23	30	34	60

```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: False
```

For example:

Test	Result
print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))	False
print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))	True

Answer: (penalty regime: 0 %)

Reset answer

```
def searchMatrix(matrix: list[list[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False

    m, n = len(matrix), len(matrix[0])
    left, right = 0, m * n - 1

    while left <= right:
        mid = (left + right) // 2
        row = mid // n
        col = mid % n
        val = matrix[row][col]

```

```
        if val == target:
            return True
        elif val < target:
            left = mid + 1
        else:
            right = mid - 1

    return False
```

	Test	Expected	Got	
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 13))	False	False	✓
✓	print(searchMatrix([[1,3,5,7],[10,11,16,20],[23,30,34,60]], 3))	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Incorrect

Mark 0.00 out of 1.00

String should contain only the words are not palindrome.

Sample Input 1

Malayalam is my mother tongue

Sample Output 1

is my mother tongue

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    input_data = sys.stdin.read().strip()
    if not input_data:
        return
    for line in input_data.splitlines():
        if not line.strip():
            continue
        words = line.split()
        non_palindromes = []
        for word in words:
            clean = "".join(ch for ch in word if ch.isalnum())
            if not clean or clean.lower() != clean.lower()[::-1]:
                non_palindromes.append(word)
        print(" ".join(non_palindromes))

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	Malayalam is my mother tongue	is my mother tongue	is my mother tongue	✓

Your code failed one or more hidden tests.
Your code must pass all tests to earn any marks. Try again.

Incorrect

Marks for this submission: 0.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array.*

Example 1:

```
Input: nums = [3,0,1]
Output: 2
Explanation: n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
```

Example 2:

```
Input: nums = [0,1]
Output: 2
Explanation: n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.
```

Example 3:

```
Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
Explanation: n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.
```

For example:

Test	Result
print(missingNumber([3,0,1]))	2
print(missingNumber([0,1]))	2

Answer: (penalty regime: 0 %)

Reset answer

```
def missingNumber(nums: list[int]) -> int:
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)
```

	Test	Expected	Got	
✓	print(missingNumber([3,0,1]))	2	2	✓
✓	print(missingNumber([0,1]))	2	2	✓
✓	print(missingNumber([9,6,4,2,3,5,7,0,1]))	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given an list, find peak element in it. A peak element is an element that is greater than its neighbors.

An element a[i] is a peak element if

A[i-1] <= A[i] >=a[i+1] for middle elements. [0<i<n-1]

A[i-1] <= A[i] for last element [i=n-1]

A[i]>=A[i+1] for first element [i=0]

Input Format

The first line contains a single integer n , the length of A .
The second line contains n space-separated integers,A[i].

Output Format

Print peak numbers separated by space.

Sample Input

5

8 9 10 2 6

Sample Output

```
10 6
```

For example:

Input	Result
4 12 3 6 8	12 8

Answer: (penalty regime: 0 %)

```
import sys

def main():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    A = [int(x) for x in input_data[1:n+1]]

    if n == 0:
        return

    peaks = []
    if n == 1:
        peaks.append(A[0])
    else:
        for i in range(n):
            if i == 0:
                if A[0] >= A[1]:
                    peaks.append(A[0])
            elif i == n - 1:
                if A[n - 1] >= A[n - 2]:
                    peaks.append(A[n - 1])
            else:
                if A[i] >= A[i - 1] and A[i] >= A[i + 1]:
                    peaks.append(A[i])

    print(*(peaks))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	7 15 7 10 8 9 4 6	15 10 9 6	15 10 9 6	✓
✓	4 12 3 6 8	12 8	12 8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
```

Example 2:

```
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 < nums[i], target < 10^4`
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

For example:

Test	Result
`print(search([-1,0,3,5,9,12],9))`	4

Answer: (penalty regime: 0 %)

Reset answer

```
from typing import List

def search(nums: List[int], target: int) -> int:
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

	Test	Expected	Got	
✓	`print(search([-1,0,3,5,9,12],9))`	4	4	✓
✓	`print(search([-1,0,3,5,9,12],2))`	-1	-1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Two string values S1, S2 are passed as the input. The program must print first N characters present in S1 which are also present in S2.

Input Format:

The first line contains S1.
The second line contains S2.
The third line contains N.

Output Format:

The first line contains the N characters present in S1 which are also present in S2.

Boundary Conditions:

2 <= N <= 10
2 <= Length of S1, S2 <= 1000

Example Input/Output 1:

Input:

abcbde
cdefghbb
3

Output:

bcd

Note:

b occurs twice in common but must be printed only once.

Answer: (penalty regime: 0 %)

```
import sys

def main():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) >= 3:
        s1 = input_data[0].strip()
        s2 = input_data[1].strip()
        n = int(input_data[2].strip())

        s2_set = set(s2)
        seen = set()
        result = []

        for char in s1:
            if char in s2_set and char not in seen:
                seen.add(char)
                result.append(char)
                if len(result) == n:
                    break

        print("".join(result))
```

```
22
23 ▾ if __name__ == '__main__':
24       main()
```

	Input	Expected	Got	
✓	abcbde cdefghbb 3	bcd	bcd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Write a Python program for binary search.

For example:

Input	Result
1,2,3,5,8 6	False
3,5,9,45,42 42	True

Answer: (penalty regime: 0 %)

```
import sys

def main():
    input_data = sys.stdin.read().splitlines()
    if len(input_data) >= 2:

        nums = [int(x.strip()) for x in input_data[0].split(',')]
        target = int(input_data[1].strip())

        nums.sort()

        left, right = 0, len(nums) - 1
        found = False

        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                found = True
                break
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        print(found)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	1,2,3,5,8 6	False	False	✓
✓	3,5,9,45,42 42	True	True	✓
✓	52,45,89,43,11 11	True	True	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.